

Prompt 1

KKC TEXT ADVENTURE — PROMPT 1 OF 180 (v6)

=====

Scaffold: Offline TypeScript MVP with Local Narration

=====

I am building a text adventure game set in the Kingkiller Chronicle universe by Patrick Rothfuss. This is a simulation-first, canon-faithful interactive fiction engine.

HARD BANS — READ BEFORE WRITING ANY CODE

Do not create, import, mention, or configure any of the following:

- Anthropic, Claude, OpenAI, GPT, or any LLM
- API keys of any kind
- fetch, axios, node-fetch, or any HTTP client
- placeholder files or comments for future API integration
- env vars other than DB_PATH and NODE_ENV
- comments like "future AI integration goes here"

If any requirement below seems to imply remote AI, replace it with a local deterministic implementation instead. The game must run fully offline from the terminal with zero network calls.

SIMPLICITY RULE

Prefer small pure functions and direct module imports.

Do not introduce service layers, manager classes, base classes, registries, factories, dependency containers, or abstractions not required by this prompt. Keep code readable, plain, and easy to extend.

ARCHITECTURE RULE

engine/* — command parsing, movement, access checks, time advancement, state mutation, persistence
narration/* — rendering text only
content/* — authored text fragments only

narration/* must never read from or write to the database.

narration/* must never mutate PlayerState.

narration/* must never look up location names or data from the DB.

repl.ts saves state exactly once per turn using the PlayerState

returned by dispatch(). Do not reload state from the DB after narration.
State must never be mutated or persisted inside narration/*.

CANON RESTRAINT RULE

If a physical detail about any University location is not clearly established in the published books, keep the description modest and generic rather than inventing precise architecture, layout, or decor.

STACK

- Node.js with TypeScript (strict mode throughout)
- SQLite via better-sqlite3
- dotenv
- readline (built-in Node — do not install as a package)
- ts-node and nodemon for development
- Jest and ts-jest for testing

package.json must include exactly these packages and no others:

dependencies:

better-sqlite3
dotenv

devDependencies:

typescript
ts-node
nodemon
jest
ts-jest
@types/jest
@types/node
@types/better-sqlite3

Do not install readline. Do not add any other packages.

DIRECTORY STRUCTURE

Create exactly this. Do not add folders or files not used in this prompt.

kkc-adventure/
src/

```
index.ts
repl.ts
types/
  index.ts
db/
  connection.ts
  schema.ts
  seed.ts
engine/
  actions.ts
  movement.ts
  state.ts
  time.ts
narration/
  provider.ts
  localNarrator.ts
  renderLocation.ts
  renderStatus.ts
content/
  locationFlavour.ts
tests/
  formatters.test.ts
data/
  .gitkeep
  .env.example
  .gitignore
  package.json
  tsconfig.json
```

TYPES — src/types/index.ts

No `any`. Use `unknown` and narrow where needed.

```
export type TimeOfDay = 'morning' | 'afternoon' | 'evening' | 'night';
export type AcademicRank = 'none' | 'E_lir' | 'Re_lar' | 'El_the';
export type CanonTier = 1 | 2 | 3; // 1=confirmed, 2=implied, 3=inferred
```

```
export interface Exit {
  direction: string;
  target_location_id: string;
  access_condition?: string; // e.g. 'requires_Re_lar', 'locked_at_night'
}
```

```
export interface Location {
```

```

id: string;
name: string;
era: string;
tier: CanonTier;
cluster_id: string;
description_base: string;
exits: Exit[];
is_accessible: boolean;
travel_time_minutes: number;
canon_source?: string;
}

```

```

export interface NPC {
  id: string;
  name: string;
  location_id: string;
  era: string;
  temperament: string;
  speech_style: string;
  conditions?: string;
}

```

```

export interface InventoryItem {
  id: string;
  name: string;
  quantity: number;
  notes?: string;
}

```

```

export interface Reputation {
  academic_standing: number;
  university_social: number;
  eolian_standing: number;
  npc_trust: Record<string, number>;
}

```

```

export interface PlayerState {
  character_id: string;
  era: string;
  location_id: string;
  money_drabs: number;
  inventory: InventoryItem[];
  reputation: Reputation;
  time_of_day: TimeOfDay;
  day_number: number;
  term_number: number;
  injuries: string[];
  hunger: number; // 0=full, 100=starving
}

```

```

    fatigue: number; // 0=rested, 100=exhausted
    academic_rank: AcademicRank;
    world_state_flags: Record<string, boolean | string | number>;
  }

export interface NarrationProvider {
  renderLocation(location: Location, state: PlayerState, npcs: NPC[], accessibleExits: Exit[]):
string;
  renderWait(state: PlayerState): string;
  renderFallback(input: string, state: PlayerState): string;
  renderHelp(): string;
}

export interface CommandResult {
  output: string;
  newState: PlayerState;
  shouldExit: boolean;
}

```

Note: NarrationProvider.renderLocation receives a pre-filtered accessibleExits array computed by the engine. The narrator never computes access rules itself.

DATABASE — src/db/connection.ts

Open the SQLite database at the path from DB_PATH in .env using better-sqlite3. Export the db instance as the default export. Throw a descriptive error if DB_PATH is not set. The connection is synchronous — better-sqlite3 requires no await.

DATABASE — src/db/schema.ts

Export runMigrations(db) which runs CREATE TABLE IF NOT EXISTS for all tables. All JSON fields are stored as TEXT and parsed at the state layer, not here.

Tables:

```

locations (
  id          TEXT PRIMARY KEY,
  name        TEXT NOT NULL,
  era         TEXT NOT NULL,

```

```

tier          INTEGER NOT NULL DEFAULT 1,
cluster_id    TEXT NOT NULL DEFAULT "",
description_base TEXT NOT NULL,
exits         TEXT NOT NULL DEFAULT '[]',
is_accessible INTEGER NOT NULL DEFAULT 1,
travel_time_minutes INTEGER NOT NULL DEFAULT 5,
canon_source  TEXT
)

```

```

npcs (
  id      TEXT PRIMARY KEY,
  name    TEXT NOT NULL,
  location_id TEXT NOT NULL,
  era     TEXT NOT NULL,
  temperament TEXT NOT NULL DEFAULT "",
  speech_style TEXT NOT NULL DEFAULT "",
  conditions TEXT
)

```

```

player_state (
  id          INTEGER PRIMARY KEY CHECK (id = 1),
  character_id TEXT NOT NULL,
  era         TEXT NOT NULL,
  location_id TEXT NOT NULL,
  money_drabs INTEGER NOT NULL DEFAULT 0,
  inventory   TEXT NOT NULL DEFAULT '[]',
  reputation  TEXT NOT NULL DEFAULT '{}',
  time_of_day TEXT NOT NULL DEFAULT 'morning',
  day_number  INTEGER NOT NULL DEFAULT 1,
  term_number INTEGER NOT NULL DEFAULT 1,
  injuries    TEXT NOT NULL DEFAULT '[]',
  hunger      INTEGER NOT NULL DEFAULT 0,
  fatigue     INTEGER NOT NULL DEFAULT 0,
  academic_rank TEXT NOT NULL DEFAULT 'E_lir',
  world_state_flags TEXT NOT NULL DEFAULT '{}'
)

```

```

world_state (
  id  INTEGER PRIMARY KEY CHECK (id = 1),
  flags TEXT NOT NULL DEFAULT '{}'
)

```

```

pending_reputation_changes (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  npc_id      TEXT NOT NULL,
  axis        TEXT NOT NULL,
  delta       INTEGER NOT NULL,
  apply_on_day INTEGER NOT NULL
)

```

)

DATABASE — src/db/seed.ts

First line of this file must be:

```
import 'dotenv/config';
```

This ensures DB_PATH is available when seed.ts is run directly.

Export runSeed(db) that calls runMigrations(db) then inserts 5 University locations using INSERT OR IGNORE.

Use only locations confirmed in the published books. Apply the canon restraint rule to all descriptions: 2-3 sentences, original prose only, grounded, sensory, present tense. Do NOT reproduce any Rothfuss line.

Locations:

```
university_mains
  name: "the Mains"
  exits:
    north -> university_artificery
    east -> university_archives_exterior
    south -> university_medica
  tier: 1, cluster_id: 'university', era: 'university'
```

```
university_artificery
  name: "the Artificery"
  exits:
    south -> university_mains
  tier: 1, cluster_id: 'university', era: 'university'
```

```
university_archives_exterior
  name: "the Archives, outer steps"
  exits:
    west -> university_mains
    enter -> university_archives_stacks
      (access_condition: 'requires_Re_lar')
  tier: 1, cluster_id: 'university', era: 'university'
```

```
university_archives_stacks
  name: "the Stacks"
  exits:
    out -> university_archives_exterior
  tier: 1, cluster_id: 'university', era: 'university'
```

```
university_medica
  name: "the Medica"
  exits:
    north -> university_mains
  tier: 1, cluster_id: 'university', era: 'university'
```

If this file is run directly (require.main === module), execute runSeed and log "Database seeded."

ENGINE — src/engine/state.ts

loadPlayerState(db): PlayerState | null
— Loads row id=1, parses all JSON fields, returns typed object or null.

savePlayerState(db, state: PlayerState): void
— Upserts row id=1, serialises all JSON fields.

initDefaultPlayerState(): PlayerState
— Returns:

```
character_id: 'kvothe'
era: 'university'
location_id: 'university_mains'
money_drabs: 300
inventory: [{ id: 'lute', name: "Kvothe's lute",
               quantity: 1,
               notes: 'Old but well-kept. Handle carefully.' }]
reputation: { academic_standing: 50, university_social: 40,
              eolian_standing: 30, npc_trust: {} }
time_of_day: 'morning'
day_number: 1
term_number: 1
injuries: []
hunger: 10
fatigue: 10
academic_rank: 'E_lir'
world_state_flags: {}
```

ENGINE — src/engine/time.ts

advanceTime(state: PlayerState, steps: number = 1): PlayerState
— Cycles morning -> afternoon -> evening -> night -> morning.
Each full cycle (night -> morning) increments day_number by 1.
Returns a new PlayerState. Do not mutate input.

timeLabel(time: TimeOfDay): string
— morning -> 'the early morning'
afternoon -> 'the afternoon'
evening -> 'the evening'
night -> 'deep in the night'

Do not add any other functions to this file.

ENGINE — src/engine/movement.ts

getLocation(db, id: string): Location | null
— Fetches location row, parses exits TEXT as Exit[], returns typed Location or null.

getAllLocations(db): Location[]
— Fetches all location rows, parses exits on each.

getNPCsAtLocation(db, location_id: string): NPC[]
— Fetches all NPC rows where location_id matches.

getAccessibleExits(location: Location, state: PlayerState): Exit[]
— Returns only exits the player can currently use.
Rules:
 'requires_Re_lar' — player academic_rank must be Re_lar or El_the
 'locked_at_night' — blocked when state.time_of_day is 'night'
Exits with no access_condition are always included.
Used for rendering only — not for movement failure logic.

movePlayer(db, state: PlayerState, direction: string):
{ success: boolean; newState: PlayerState; message: string }

movePlayer() must distinguish between two failure cases:
(a) no exit exists in the requested direction
(b) an exit exists but is currently blocked by access_condition

Implementation steps, in order:

1. Load the current location from the database.
2. Search location.exits (the FULL unfiltered list) for an exit whose direction matches the input (case-insensitive).
3. If no matching exit exists:
 return success: false, unchanged state,
 message: "You cannot go that way."
4. If a matching exit exists, check its access_condition:
 'requires_Re_lar' and player rank is E_lir or none:
 return success: false, unchanged state,

message: "The Stacks are closed to you. Only a Re'lar may enter the Archives unescorted."

'locked_at_night' and time_of_day is 'night':

return success: false, unchanged state,

message: "That way is shut for the night."

5. If the exit exists and passes all access checks:

return success: true, newState with updated location_id,

message: "" (empty — caller constructs the arrival line)

No game jargon in any failure message. In-world language only.

Do not use getAccessibleExits() inside movePlayer().

ENGINE — src/engine/actions.ts

```
dispatch(  
  input: string,  
  state: PlayerState,  
  db,  
  narrator: NarrationProvider  
): Promise<CommandResult>
```

Rules:

- Trim whitespace and normalise case before parsing.
- Engine owns all command parsing and state changes.
- Narrator receives resolved state and pre-computed data.

Commands:

look / look around

- > load current location and NPCs from DB
- > compute accessibleExits via getAccessibleExits()
- > return narrator.renderLocation(location, state, npcs, accessibleExits)
- > state unchanged

go [direction] / bare direction words

(north south east west up down enter out)

-> call movePlayer()

-> on success:

load new location and NPCs from DB

compute accessibleExits for new location

prepend a short plain arrival line to the output, e.g.:

"You make your way north."

The arrival line must be short and plain.

Do not include the destination name in the arrival line.

Append narrator.renderLocation() for the new location.

-> on failure:

return the failure message from movePlayer(), state unchanged

wait

- > call advanceTime()
- > load current location and NPCs from DB
- > compute accessibleExits
- > return narrator.renderWait(newState)
+ narrator.renderLocation(location, newState, npcs, accessibleExits)

status / stats

- > return renderStatus(state), state unchanged

inventory / inv / i

- > return renderInventory(state), state unchanged

help

- > return narrator.renderHelp(), state unchanged

quit / exit

- > return shouldExit: true with a brief in-world farewell line

anything else

- > return narrator.renderFallback(input, state), state unchanged

Return: { output: string; newState: PlayerState; shouldExit: boolean }

CONTENT — src/content/locationFlavour.ts

Export LOCATION_FLAVOUR: Record<string, string[]>

Each location_id maps to an array of exactly 4 short atmospheric lines.
Original prose only — no Rothfuss quotes. Apply canon restraint rule.
Grounded, sensory, 1-2 sentences each.

Provide entries for all 5 seeded locations.

NARRATION — src/narration/provider.ts

Re-export NarrationProvider from src/types/index.ts.
No other logic in this file.

NARRATION — src/narration/renderLocation.ts

```
renderLocation(
  location: Location,
  state: PlayerState,
  npcs: NPC[],
  accessibleExits: Exit[]
): string
```

This function receives `accessibleExits` pre-computed by the engine. It does not query the database. It does not call `getAccessibleExits()`.

Returns a formatted multi-line string:

1. Location name as a header line.
Use the location's stored name with only the first letter capitalised. Do not apply any other title-casing.
Example: stored name "the Mains" -> header "--- The Mains ---"
2. `description_base`
3. One flavour line from `LOCATION_FLAVOUR`, chosen deterministically:

$$\text{index} = (\text{state.day_number} + \text{timeIndex}(\text{state.time_of_day})) \% \text{flavourLines.length}$$
`timeIndex`: morning=0, afternoon=1, evening=2, night=3
 No `Math.random()`. Same state always produces the same line.
 If no flavour lines exist for this location, omit silently.
4. "It is [`timeLabel(state.time_of_day)`]."
5. Exits rendered using directions only — no destination names, because the narration layer must not query the database.
Build this line from the `accessibleExits` array only.
Do not mention exits that are not in `accessibleExits`.
Examples:
 "You could go north or east."
 "You could go south."
 "There is no obvious way out."
6. NPCs present if any: "Simmon is here."
If none, omit this line entirely.

NARRATION — `src/narration/renderStatus.ts`

Implement and export:

```
formatCurrency(drabs: number): string
```

— 300 -> "3 talents"
125 -> "1 talent, 2 jots, 5 drabs"
15 -> "1 jot, 5 drabs"
7 -> "7 drabs"
0 -> "nothing"
Rates: 10 drabs = 1 jot, 10 jots = 1 talent.
Omit zero-value denominations.

hungerLabel(hunger: number): string
fatigueLabel(fatigue: number): string
— 0-20: well-fed / rested
21-50: peckish / tired
51-80: hungry / weary
81-100: starving / exhausted

renderStatus(state: PlayerState): string
— Multi-line block:
Character and era
Current location (location_id)
Money (formatted)
Academic rank
Time of day, day number, term number
Hunger and fatigue labels
Injuries listed simply (if any)

renderInventory(state: PlayerState): string
— Clean list of inventory items with quantities and notes.
If empty: "You are carrying nothing of note."

NARRATION — src/narration/localNarrator.ts

Implement LocalNarrator as a class implementing NarrationProvider.

LocalNarrator:

- does NOT parse commands
- does NOT advance time
- does NOT call getAccessibleExits()
- does NOT save to or read from the database
- does NOT mutate PlayerState
- does NOT accept db as a constructor argument
- renders text only

Methods:

renderLocation(location, state, npcs, accessibleExits): string
— delegates to renderLocation() from renderLocation.ts

renderWait(state: PlayerState): string
— Brief in-world line about time passing.
Pool of exactly 4 lines. Choose deterministically:
index = state.day_number % 4

renderFallback(input: string, state: PlayerState): string
— Pool of exactly 5 short atmospheric lines.
Choose deterministically: index = input.length % 5
Example: "The thought passes without resolution."

renderHelp(): string
— Short in-world list of available actions.
Written as if a fellow student is giving practical advice.
No numbered menus. No game jargon.

REPL — src/repl.ts

startREPL(db, narrator: NarrationProvider): void

- Use Node's built-in readline module.
- Load player state. If none exists, call `initDefaultPlayerState()` and save it immediately.
- Print this welcome text as a fixed string (not pooled or generated):
Write a fixed 2-4 sentence welcome in second person, present tense.
Kvothe waking to another University morning. Original prose only.
Do NOT quote Rothfuss. Keep it modest and atmospheric.
- Before the first `renderLocation()` call, explicitly load the starting location and NPCs from the database.
- Compute `accessibleExits` for the starting location.
- Print `renderLocation()` for the starting location.
- Show prompt: "> "
- On each line of input:
call `dispatch()`
print output
if `shouldExit`: close readline and return
otherwise call `savePlayerState(db, result.newState)`
- Handle readline close event gracefully.
- State is saved exactly once per turn here and nowhere else.

ENTRY POINT — src/index.ts

First line must be:

```
import 'dotenv/config';
```

Then in order:

```
import db from './db/connection';
import { runMigrations } from './db/schema';
import { runSeed } from './db/seed';
import { LocalNarrator } from './narration/localNarrator';
import { startREPL } from './repl';
```

```
runMigrations(db);
runSeed(db);
const narrator = new LocalNarrator();
startREPL(db, narrator);
```

CONFIG FILES

package.json scripts:

```
"dev": "nodemon --watch src --ext ts --exec \"ts-node src/index.ts\""
"build": "tsc"
"start": "node dist/index.js"
"db:seed": "ts-node -r dotenv/config src/db/seed.ts"
"test": "jest"
```

Include inline Jest config in package.json:

```
"jest": {
  "preset": "ts-jest",
  "testEnvironment": "node",
  "roots": ["<rootDir>/tests"]
}
```

tsconfig.json:

```
target:      ES2020
module:      CommonJS
strict:      true
outDir:      ./dist
rootDir:     ./src
sourceMap:   true
resolveJsonModule: true
esModuleInterop: true
```

.env.example:

```
DB_PATH=./data/game.db
NODE_ENV=development
```

.gitignore:

node_modules/
dist/
data/*.db
.env

TESTS — tests/formatters.test.ts

Import formatCurrency from src/narration/renderStatus.ts.
Import advanceTime from src/engine/time.ts.
Import movePlayer, getLocation from src/engine/movement.ts.
Import runMigrations from src/db/schema.ts.
Import runSeed from src/db/seed.ts.

formatCurrency tests:

1. formatCurrency(300) -> "3 talents"
2. formatCurrency(15) -> "1 jot, 5 drabs"
3. formatCurrency(7) -> "7 drabs"
4. formatCurrency(0) -> "nothing"

advanceTime tests:

5. morning + 1 step -> afternoon, day_number unchanged
6. night + 1 step -> morning, day_number incremented by 1

movePlayer tests:

Use an in-memory SQLite database via better-sqlite3 for these tests.
Open it with: new Database(':memory:')
Call runMigrations() and runSeed() on it before the tests run.
Use initDefaultPlayerState() for the base player state.

7. Moving north from university_mains returns success: true and newState.location_id === 'university_artificery'
8. Moving west from university_mains returns success: false and message === "You cannot go that way."
9. Moving enter from university_archives_exterior as E_lir returns success: false and message containing "Re'lar"

TERMINAL COMMANDS

Provide exact WSL terminal commands to:

1. Create the project folder and initialise it
2. Install all dependencies
3. Run the dev server
4. Run the seed script independently
5. Run the tests

FINAL CONSTRAINTS

- No `any` types anywhere
- No external API calls, no HTTP, no network
- No web server, no frontend
- No unused imports or files
- No unused functions — if a function is defined, it must be called somewhere in this prompt's codebase
- No future-facing placeholder files or comments
- All files compile cleanly under strict TypeScript
- Comments are purposeful — not decorative, not aspirational

SUCCESS CRITERIA

This prompt is complete when:

- `look` prints a location description with directions only
- `go north` moves to an adjacent location with a plain arrival line
- `go east` then `enter` at the Archives returns an in-world blocked message containing "Re'lar"
- `go west` from a location with no west exit returns "You cannot go that way."
- `status` shows money, rank, hunger, fatigue
- `inventory` shows the lute
- `wait` advances time and re-renders the location
- `quit` exits cleanly
- All 9 tests pass
- No network calls occur at any point
- State is saved exactly once per turn in repl.ts
- narration/* never mutates or persists state
- movePlayer() correctly distinguishes a blocked exit from a non-existent exit using the full exit list
- A future NarrationProvider implementation can replace LocalNarrator without touching engine/, db/, repl.ts, or types/

=====
END OF PROMPT 1 v6
=====